AI-Driven Safe Dynamic Network Slicing for SLA-Preserving SDN Networks

Project Guide: Dr. Yoganand S

Team Members

Preetham (24BCE2620)

Srihari Reddy (24BCI0088)

Nath Bishal (24BKT0123)

Course: Computer Networks Project

Abstract

- Static bandwidth allocation cannot follow changing traffic demand, wasting capacity or starving latency-critical flows.
- Software-Defined Networking (SDN) separates control from forwarding, so a controller can program switches and re-allocate resources centrally.
- The project implements an AI-driven approach (SRA-LinUCB) that chooses dynamic resource-allocation actions for three slices: URLLC, eMBB and Best Effort.
- A SLA and security guardrail validates every AI action before the Ryu controller executes it.
- Environment: Mininet, Open vSwitch, Ryu (OpenFlow 1.3), iperf3 on WSL2 Ubuntu.
- Current status: all nine modules (01 to 09) are implemented and verified. The end-to-end AI workflow is operational, from telemetry and context through SRA-LinUCB, the guardrail and Ryu control to feedback and evaluation.

9 modules in the project

9 implemented and verified (01 to 09)

3 slices: URLLC, eMBB, Best Effort

Introduction and Motivation

- Networks carry traffic with very different needs: some flows need low delay, others high throughput.
- Fixed bandwidth shares are inefficient: idle capacity in one class cannot help another.
- Low-latency traffic needs protection, because bulk transfers queue ahead of it on shared links.
- Network slicing separates traffic classes into logical slices with their own targets.
- Dynamic allocation adapts shares to demand, but an AI decision can be wrong, so it needs safety validation before it is applied.

Critical communication (URLLC)

Control or alarm messages: tiny, but need very low delay.

High-bandwidth video (eMBB)

Streaming or video calls: need high throughput.

Ordinary internet (Best Effort)

Browsing and downloads: no strict guarantee.

Problem Statement

Static allocation

Fixed shares cannot respond to changing demand. Capacity sits idle in one slice while another is congested.

Unrestricted dynamic allocation

An AI that reallocates freely may take bandwidth from URLLC and violate its SLA, or ignore security constraints.

Central problem: adapt slice bandwidth to live traffic without breaking the SLA of critical traffic or security constraints.

Implemented approach: AI proposes an action, the guardrail validates it, the Ryu controller executes it, and feedback updates the AI.

Project Objectives and Status

#	Objective	Status
1	SDN-based network-slicing framework	**Implemented and Verified**
2	URLLC, eMBB, Best Effort as separate slices (host pairs H1-H4, H2-H5, H3-H6)	**Implemented and Verified**
3	Collect telemetry and generate context	**Implemented and Verified**
4	SRA-LinUCB resource-allocation decisions	**Implemented and Verified**
5	Validate actions against SLA and security constraints	**Implemented and Verified**
6	Evaluate utilization, performance and safety	**Implemented and Verified**
7	Compare static, unsafe dynamic, safe dynamic	**Implemented and Verified**

All seven objectives are implemented and verified. Objective 2: the three traffic classes map to host pairs H1-H4, H2-H5 and H3-H6, which share the S1-S2 bottleneck link.

Research Gap

Static slicing

Predictable and simple, but wastes capacity when demand shifts.

Learning-based slicing

Adapts to demand, but unconstrained exploration can break SLA targets.

This project

Combines adaptive allocation with a validation layer for SLA and security.

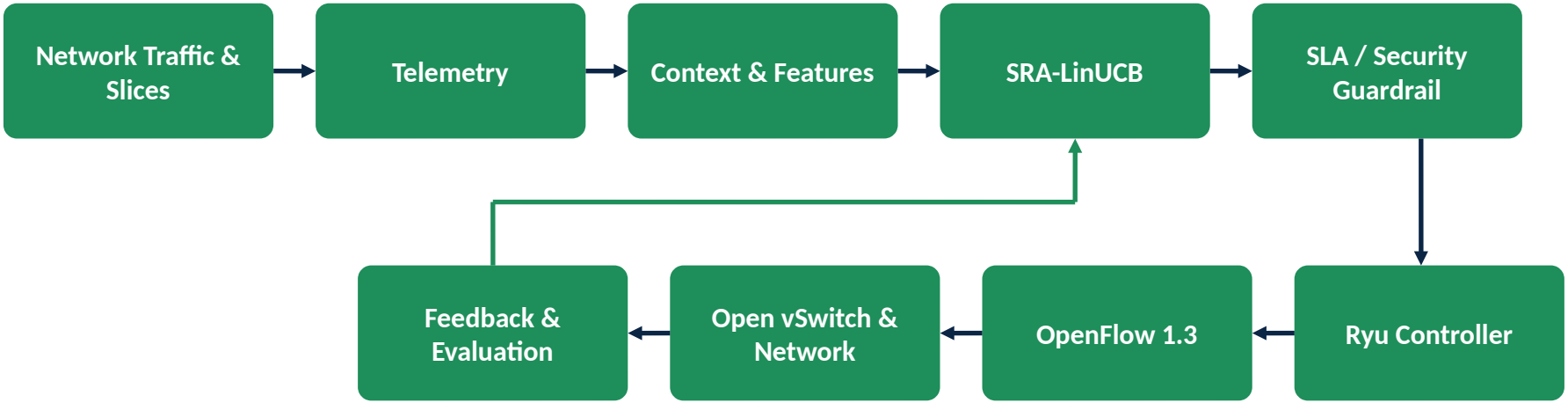
- Gap addressed: adaptive allocation that is explicitly checked before execution.
- Positioning is general background; this report does not include a literature survey or citations.

Implementation Status: Nine Modules

No.	Module	Purpose	Status
01	SDN Network & Slicing	Mininet + OVS topology, traffic slices	**Implemented and Verified**
02	Network Telemetry	Latency, throughput, loss, queue/utilization	**Implemented and Verified**
03	Context & Feature	Telemetry to AI input	**Implemented and Verified**
04	SRA-LinUCB AI	Select resource-allocation actions	**Implemented and Verified**
05	Security	Protection and overhead measurement	**Implemented and Verified**
06	SLA & Security Guardrail	Validate AI actions before execution	**Implemented and Verified**
07	SDN Control	Ryu + OpenFlow control of OVS	**Implemented and Verified**
08	Feedback & Learning	Update AI from network results	**Implemented and Verified**
09	Evaluation & Comparison	Static vs dynamic vs safe-dynamic	**Implemented and Verified**

All nine modules are implemented, integrated and verified through the end-to-end workflow.

System Architecture and Data Flow



SRA-LinUCB proposes an action from the context; the guardrail validates it against SLA and security constraints; Ryu executes only approved actions on Open vSwitch over OpenFlow 1.3. Feedback returns performance results to SRA-LinUCB, closing the learning loop.

Implemented and Verified

All components are implemented and verified; the flow follows the project data flow.

Tools and Implementation Environment

Tool	Version	Role
Python (pyenv env ryu-3.9)	3.9.18	Ryu runtime and project code
Ryu	4.34	SDN controller framework
Mininet	2.3.0	Network emulation
Open vSwitch	3.3.9	Virtual OpenFlow switches
OpenFlow	1.3	Controller-switch protocol
iperf3	3.16	Traffic generation tool
WSL2 Ubuntu + VS Code + Git	-	Development and version control

Mininet, OVS and Ryu must run inside the WSL Ubuntu environment (README).

Repository Structure (Verified)

```
AI-Safe-Dynamic-Slicing/
  README.md
  requirements.txt
  .gitignore
  .python-version
  ai/
  config/
  controller/    ryu_controller.py
  data/
  experiments/
  guardrail/
  results/
  security/
  slicing/
  telemetry/
  topology/      topology.py
```

Core and network modules

config/, controller/, experiments/, slicing/, telemetry/ and topology/ hold the configuration, Ryu control, experiment, slicing, telemetry and Mininet topology code. Two source files are named in the README tree.

AI, security, guardrail and results

ai/, security/, guardrail/, data/ and results/ complete the README layout for the SRA-LinUCB, security, guardrail, data and evaluation-result components.

Module 01: SDN Network and Slicing



H1 - H4: URLLC | **H2 - H5: eMBB** | **H3 - H6: Best Effort**

Implemented and Verified

Mininet + OVS; S1-S2 is the shared bottleneck link. Verified with pingall: 0% dropped.

Module 07: SDN Control (Ryu + OpenFlow 1.3)

Ryu app
controller/ryu_controller.py

→

OpenFlow 1.3
Controller-switch channel

→

Open vSwitch
S1 and S2 forward traffic

- Behaviour: the Ryu app installs OpenFlow 1.3 forwarding on S1 and S2, including MAC-learning forwarding for host connectivity.
- Verified with pingall in Mininet: 0% dropped.
- Input: guardrail-approved resource-allocation actions. Processing: each approved action is translated into OpenFlow 1.3 messages. Output: updated configuration on Open vSwitch.
- Role in the complete system: executes guardrail-approved actions on OVS; telemetry (Module 02) observes the result and feedback (Module 08) updates SRA-LinUCB.

Implemented and Verified

Modules 02 and 03: Telemetry and Context

Module 02: Network Telemetry

Collects latency, throughput, loss and queue/utilization from the running network. Input: live slice traffic on the Mininet + OVS network. Output: telemetry passed to Module 03 and to the feedback path.

Module 03: Context & Feature

Converts telemetry into the input (context) used by the AI. Input: Module 02 telemetry. Output: the context features that SRA-LinUCB (Module 04) uses to select an action.

- The telemetry/ folder holds Module 02; it feeds Module 03 and the feedback path (Module 08).
- No telemetry values or feature vectors are tabulated in this report; it documents implementation and verification status.

Implemented and Verified

Module 04: SRA-LinUCB

- Purpose: selects dynamic resource-allocation actions for the URLLC, eMBB and Best Effort slices.
- Status: implemented and verified as part of the end-to-end workflow.
- LinUCB is a contextual bandit: it scores each action from the context and adds an exploration bonus.
- Input: context from Module 03. Output: a proposed action sent to the guardrail (Module 06). Feedback from Module 08 updates the model.

Standard LinUCB (background)

$$a^* = \arg\max_a \left(x^T \theta_a + \alpha \sqrt{x^T A_a^{-1} x} \right)$$

x: context θ_a: estimated weights α: exploration

Implemented and Verified

Formula is the standard LinUCB selection rule on which SRA-LinUCB is based.

Modules 05 and 06: Security and Guardrail



Module 06: SLA & Security Guardrail

Validates every AI action against SLA and security constraints before execution. Input: action proposed by SRA-LinUCB. Output: approved action for Ryu. Implemented and verified.

Module 05: Security

Applies security protection and measures its overhead, so the guardrail accounts for security cost when validating actions. Implemented and verified.

Diagram shows the implemented flow: every AI action passes through the guardrail before the Ryu controller executes it on Open vSwitch.

Modules 08 and 09: Feedback and Evaluation

Scenario	Allocation	Guardrail	Evaluation status
Static	Fixed slice shares	None	**Implemented and Verified**
Unsafe dynamic	AI-driven	None	**Implemented and Verified**
Safe dynamic	AI-driven	SLA + security validation	**Implemented and Verified**

- Evaluation metrics: latency, throughput, loss and queue/utilization, collected by Module 02.
- Feedback (Module 08) returns network results to SRA-LinUCB so it learns from each applied action.
- The three-scenario comparison (Module 09) is implemented and verified; this report does not state quantitative performance values.

Implemented and Verified

Execution Workflow

Terminal 1: Ryu controller

```
cd ~/AI-Safe-Dynamic-Slicing
pyenv activate ryu-3.9
ryu-manager controller/ryu_controller.py
```

Terminal 2: Mininet topology

```
cd ~/AI-Safe-Dynamic-Slicing
pyenv activate ryu-3.9
sudo mn -c        # if old instance
sudo "$(pyenv which python)" \
  topology/topology.py
```

Tests inside mininet>

```
pingall
h1 ping -c 3 h4
h2 ping -c 3 h5
h3 ping -c 3 h6
```

- Start the controller first and keep it running.
- Expected from pingall: 0% dropped.
- Run everything inside WSL Ubuntu.

Testing and Verification

Test	Evidence	Status
pingall connectivity	0% dropped in Mininet	**Verified**
Per-path pings (H1-H4, H2-H5, H3-H6)	h1 ping -c 3 h4, h2 ping -c 3 h5, h3 ping -c 3 h6	**Verified**
Congestion test (iperf3)	Heavy eMBB traffic can raise URLLC latency; no numerical values are reported	**Verified**
Module and end-to-end verification	Modules 01 to 09 run through the Ryu + Mininet workflow	**Verified**
Terminal evidence	pingall, per-path ping and iperf3 congestion run output	**Verified**

Verification covers the software implementation and its functional behaviour. Quantitative performance values are not claimed in this report.

Limitations and Scope

Limitations

- Runs on emulation (Mininet), not hardware.
- Topology has one shared bottleneck link (S1-S2) and three host pairs.
- This report presents implementation and verification status; quantitative comparisons are not reported.

Optional future enhancements

- Evaluate on physical OpenFlow hardware.
- Extend the topology beyond a single bottleneck link.
- Add measured static, unsafe-dynamic and safe-dynamic comparison tables and graphs.

Conclusion and Future Work

- Implemented and verified: all nine modules, from the Mininet + OVS topology with three traffic classes (01) and telemetry, context, SRA-LinUCB and security (02 to 05) to the guardrail, Ryu OpenFlow 1.3 control, feedback and evaluation (06 to 09).
- Integrated architecture: traffic flows through slicing, telemetry, context generation and SRA-LinUCB; the SLA and security guardrail validates each action before Ryu applies it to Open vSwitch; feedback closes the loop.
- Verified: pingall with 0% dropped, per-path pings for H1-H4, H2-H5 and H3-H6, and the preliminary congestion check.
- Optional future enhancements: evaluation on physical hardware and larger topologies, and measured comparisons of static, unsafe dynamic and safe dynamic allocation.